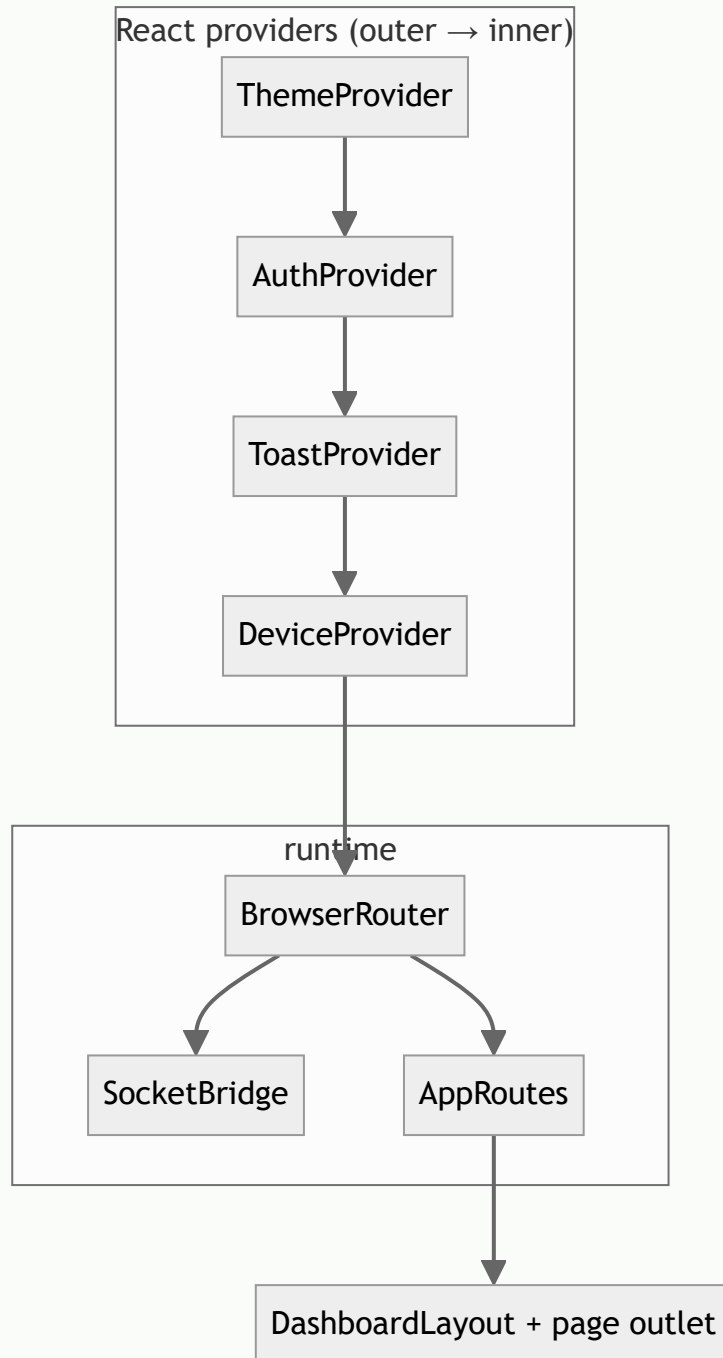


Web frontend reference

React dashboard for Smart AgriTech EMS: `web_frontend/`.

Built with **Vite**, **React 18**, **React Router**, **Tailwind CSS**, and **Recharts**. Served in production as static files behind **nginx** (Docker image).

Application shell



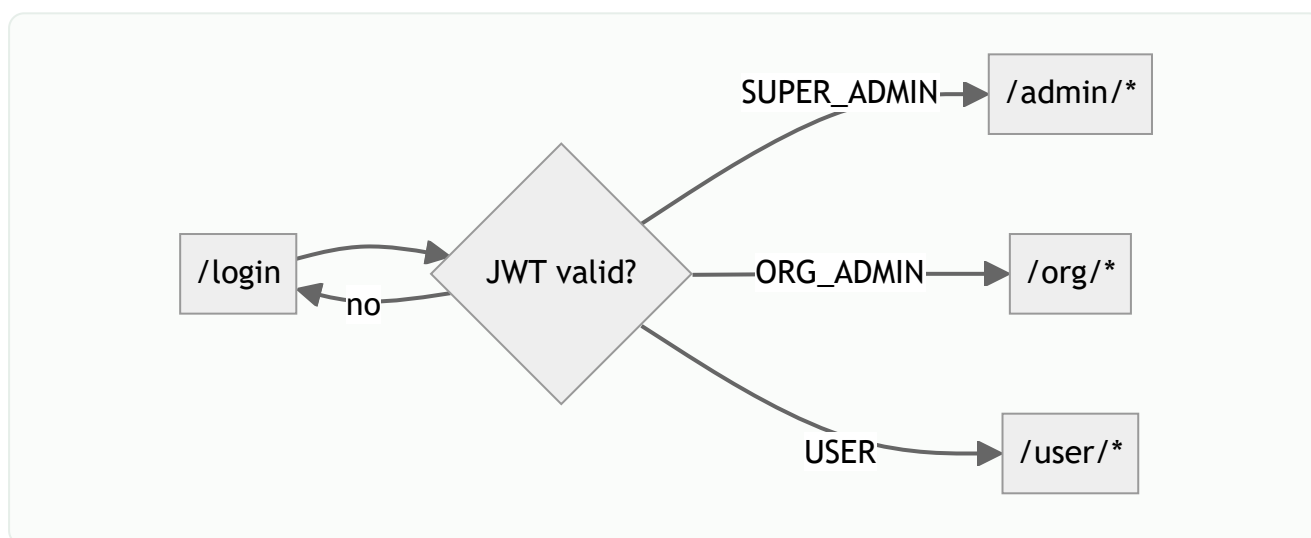
Provider	Responsibility
ThemeContext	Light/dark mode, org theme colors from API
AuthContext	JWT session, login/logout, role mapping
ToastContext	Global success/error toasts
DeviceContext	Selected device/slave, device list cache

Provider	Responsibility
SocketBridge	Connects Socket.IO when logged in; refreshes devices on <code>reading:new</code>

Role-based routing

Backend roles map to URL prefixes via `utils/roles.js` :

Backend enum	Frontend path	Nav config
<code>SUPER_ADMIN</code>	<code>/admin</code>	<code>adminNav</code>
<code>ORG_ADMIN</code>	<code>/org</code>	<code>orgNav</code>
<code>USER</code>	<code>/user</code>	<code>userNav</code>



`ProtectedRoute` redirects unauthenticated users to `/login` and wrong-role users to their home prefix.

Complete route map

Public

Path	Page	Purpose
<code>/login</code>	<code>Login.jsx</code>	Email/password login
<code>/</code>	redirect	→ role home or <code>/login</code>

Super Admin — `/admin`

Path	Component	Functionality
<code>/admin</code>	<code>AdminDashboard</code>	Fleet KPIs, charts, device summary
<code>/admin/dashboard-detail</code>	<code>DashboardDetailPage</code>	Live variable tiles for selected device
<code>/admin/organizations</code>	<code>AdminOrganizations</code>	CRUD organizations
<code>/admin/users</code>	<code>AdminUsers</code>	CRUD all users
<code>/admin/gateways</code>	<code>AdminGateways</code>	Gateway CRUD
<code>/admin/devices</code>	<code>AdminDevices</code>	Device list, create, assign template
<code>/admin/devices/:deviceId</code>	<code>DeviceDetailPage</code>	Slaves, variables, users, MQTT modal
<code>/admin/device-templates</code>	<code>AdminDeviceTemplates</code>	Template library
<code>/admin/device-templates/:templateId</code>	<code>TemplateDetailPage</code>	Slaves + variables editor
<code>/admin/icons</code>	<code>AdminManageIcons</code>	Icon asset management
<code>/admin/products</code>	<code>AdminProducts</code>	Product catalog
<code>/admin/data-center</code>	<code>AdminDataCenter</code>	Bulk data operations
<code>/admin/sensor-history</code>	<code>SensorHistoryPage</code>	Raw ingest log, pagination
<code>/admin/historical-data</code>	<code>AdminHistoricalData</code>	Multi-variable charts + CSV
<code>/admin/variable-alarms</code>	<code>AdminVariableAlarms</code>	Active variable alarm states
<code>/admin/alarm-history</code>	<code>AlarmHistoryPage</code>	Historical alarms (device filter)
<code>/admin/linkage-records</code>	<code>AdminLinkageRecords</code>	Alarm linkage audit
<code>/admin/template-triggers</code>	<code>AdminTemplateTriggers</code>	Alarm trigger templates
<code>/admin/alarm-settings</code>	<code>AdminAlarmSettings</code>	Per-device alarm thresholds
<code>/admin/alarm-contacts</code>	<code>AdminAlarmContacts</code>	Notification contacts
<code>/admin/device-timestamps</code>	<code>AdminDeviceTimestamps</code>	Connectivity events
<code>/admin/schedule-tasks</code>	<code>AdminScheduleTasks</code>	Cron-style device switches
<code>/admin/theme-settings</code>	<code>AdminThemeSettings</code>	Global UI themes
<code>/admin/settings</code>	<code>AdminSettings</code>	System settings

Organization Admin — `/org`

Path	Component	Functionality
<code>/org</code>	<code>OrgDashboard</code>	Org-scoped dashboard
<code>/org/dashboard-detail</code>	<code>DashboardDetailPage</code>	Live device detail
<code>/org/users</code>	<code>OrgUsers</code>	Team users in org
<code>/org/widget-templates</code>	<code>OrgWidgetTemplates</code>	Dashboard widget layouts
<code>/org/devices</code>	<code>OrgDevices</code>	Org device management
<code>/org/devices/:deviceId</code>	<code>DeviceDetailPage</code>	Device config (org scope)
<code>/org/gateways</code>	<code>OrgGateways</code>	Org gateways
<code>/org/device-templates</code>	<code>OrgDeviceTemplates</code>	Org templates
<code>/org/device-templates/:templateId</code>	<code>TemplateDetailPage</code>	Template editor
<code>/org/sensor-history</code>	<code>SensorHistoryPage</code>	Sensor logs
<code>/org/historical-data</code>	<code>OrgHistoricalData</code>	Historical charts
<code>/org/device-timestamps</code>	<code>OrgDeviceTimestamps</code>	Connectivity log
<code>/org/template-triggers</code>	<code>OrgTemplateTriggers</code>	Alarm triggers
<code>/org/alarm-settings</code>	<code>OrgAlarmSettings</code>	Alarm config
<code>/org/alarm-contacts</code>	<code>OrgAlarmContacts</code>	Contacts
<code>/org/alarm-history</code>	<code>AlarmHistoryPage</code>	Alarm log
<code>/org/schedule-tasks</code>	<code>OrgScheduleTasks</code>	Schedules
<code>/org/settings</code>	<code>OrgSettings</code>	Org preferences

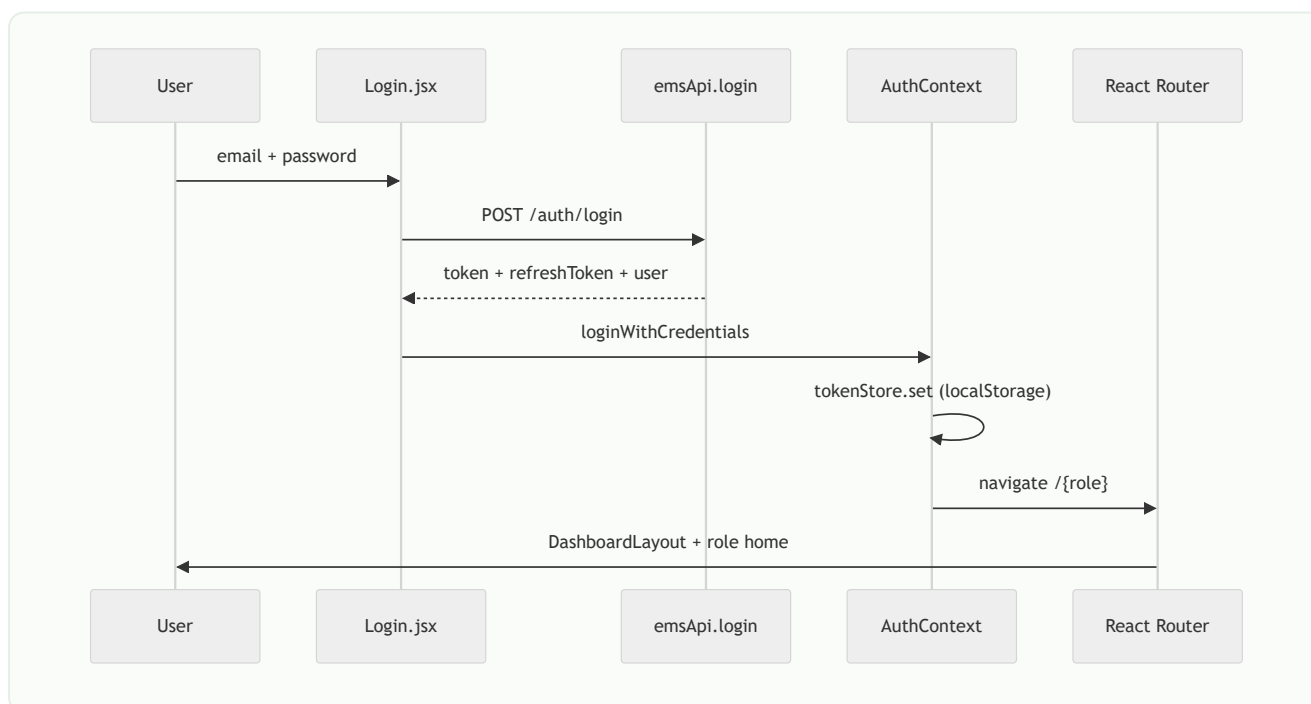
End User — `/user`

Path	Component	Functionality
<code>/user</code>	<code>UserDashboard</code>	Assigned devices overview
<code>/user/dashboard-detail</code>	<code>DashboardDetailPage</code>	Live readings
<code>/user/account</code>	<code>UserAccountSettings</code>	Profile, password
<code>/user/notifications</code>	<code>UserNotifications</code>	In-app notifications
<code>/user/subscription</code>	<code>UserSubscription</code>	Plan request
<code>/user/products</code>	<code>UserProducts</code>	Browse products

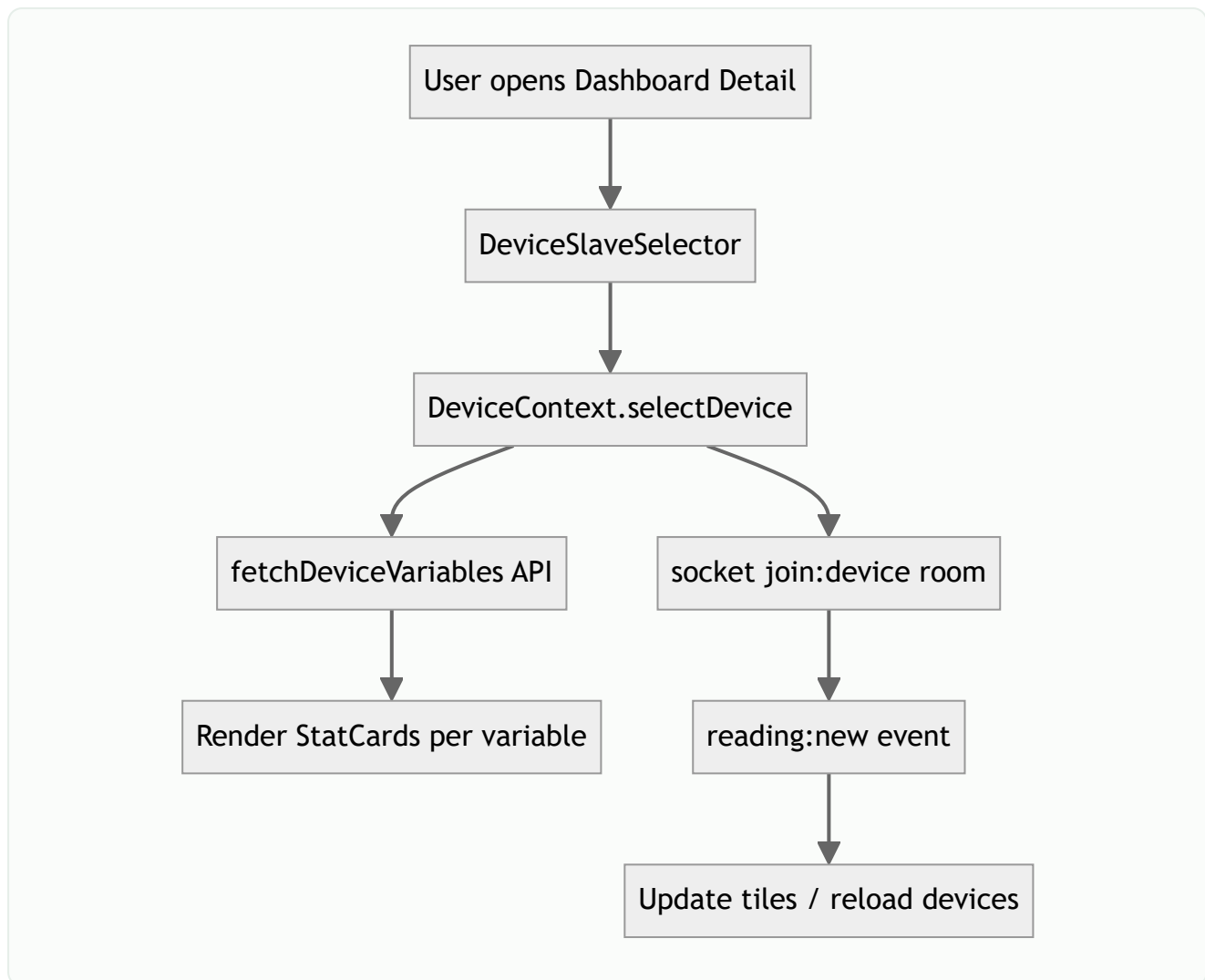
Path	Component	Functionality
/user/schedule	UserSchedule	View schedules
/user/slab-rates	UserSlabRates	Electricity tariff slabs
/user/interval-history	UserIntervalHistory	Billing intervals
/user/sensor-history	SensorHistoryPage	Sensor history
/user/alarm-template	UserAlarmTemplate	User alarm templates
/user/alarm-settings	UserAlarmSettings	User alarm config
/user/alarm-history	AlarmHistoryPage	Alarm history
/user/ai-analytics	UserAIAnalytics	AI analytics hub
/user/voltage-imbalance	UserVoltageImbalance	Phase voltage analytics
/user/current-imbalance	UserCurrentImbalance	Current imbalance
/user/power-factor	UserPowerFactor	Power factor trends
/user/energy-consumption	UserEnergyConsumption	Consumption charts
/user/anomalies	UserAnomalies	Anomaly timeline

User journey diagrams

Login → dashboard



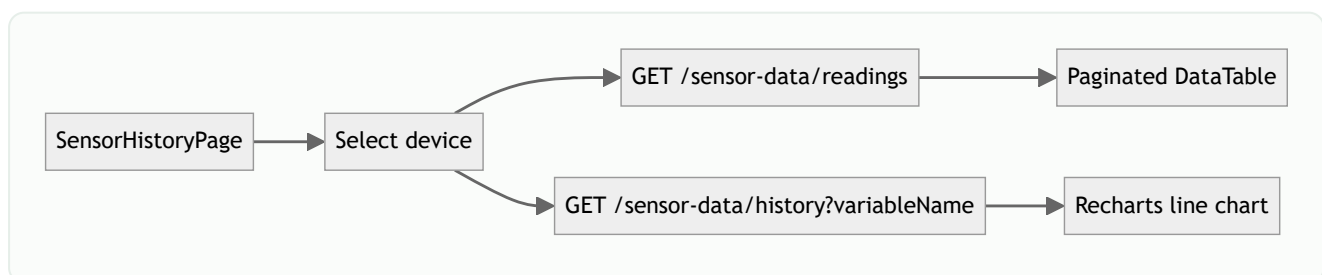
Device selection → live readings



Shared pages (`DashboardDetailPage` , `SensorHistoryPage` , `AlarmHistoryPage`) use:

- `DeviceSlaveSelector` — pick device (and optional Modbus slave)
- `utils/sensorReadings.js` — `fetchDeviceVariables` , `fetchDeviceDashboardCharts` , `latestToReadings`
- `utils/dashboardHelpers.js` — chart aggregation helpers

Sensor history flow



Note: History queries omit `slaveId` when raw ingest rows have null `deviceConfigSlaveId` (common for single-slave devices).

Alarm history flow



API client layer

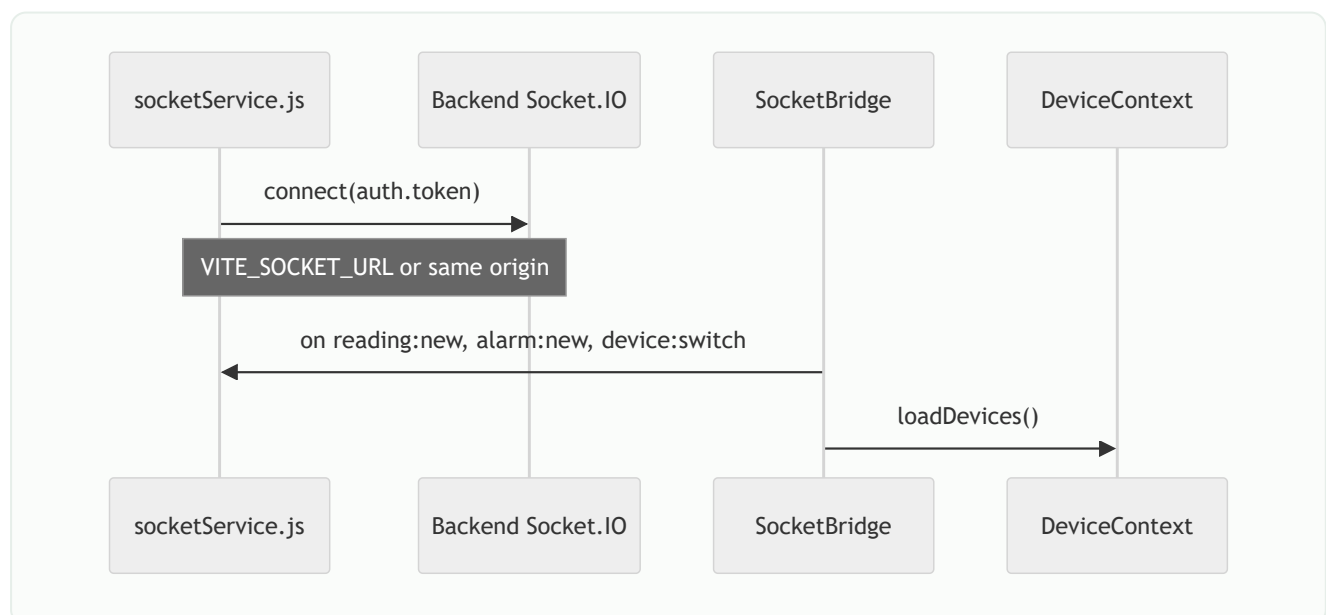
`api/client.js`

Feature	Behavior
Base URL	<code>VITE_API_URL</code> (build-time) or <code>/api</code> (dev proxy)
Auth header	<code>Authorization: Bearer <token></code>
401 handling	Auto refresh via <code>/auth/refresh</code> ; logout on failure
Errors	<code>ApiError</code> with HTTP status

`api/emsApi.js`

Central facade for all REST calls grouped by domain (auth, devices, sensor-data, alarms, AI, etc.). Pages import `emsApi` rather than raw `fetch`.

Real-time (Socket.IO)



Env var	Production value
VITE_API_URL	https://iotbackend.domain.com/api
VITE_SOCKET_URL	https://iotbackend.domain.com

CapRover: enable **WebSocket Support** on the backend app.

Layout & UI components

```
web_frontend/src/
├── components/
│   ├── layout/      DashboardLayout, Sidebar, Topbar
│   ├── shared/      DeviceSlaveSelector, TimeRangeChips
│   ├── ui/          Modal, DataTable, StatCard, FormFields, PageState
│   └── SocketBridge.jsx
├── pages/
│   ├── admin/       Super-admin screens
│   ├── org/         Org-admin screens
│   ├── user/        End-user screens
│   └── shared/       Cross-role pages
├── context/         Auth, Theme, Device, Toast
├── config/navConfig.jsx
└── utils/           roles, mappers, sensorReadings, dashboardHelpers
```

`DashboardLayout` renders sidebar from `navConfig`, top bar (user menu, theme toggle), and `<Outlet />` for nested routes.

Build & environment

Local development

```
cd web_frontend
npm install
npm run dev    # http://localhost:5173
```

Vite dev server proxies `/api` to `http://localhost:5000` (see `vite.config.js`).

Production build

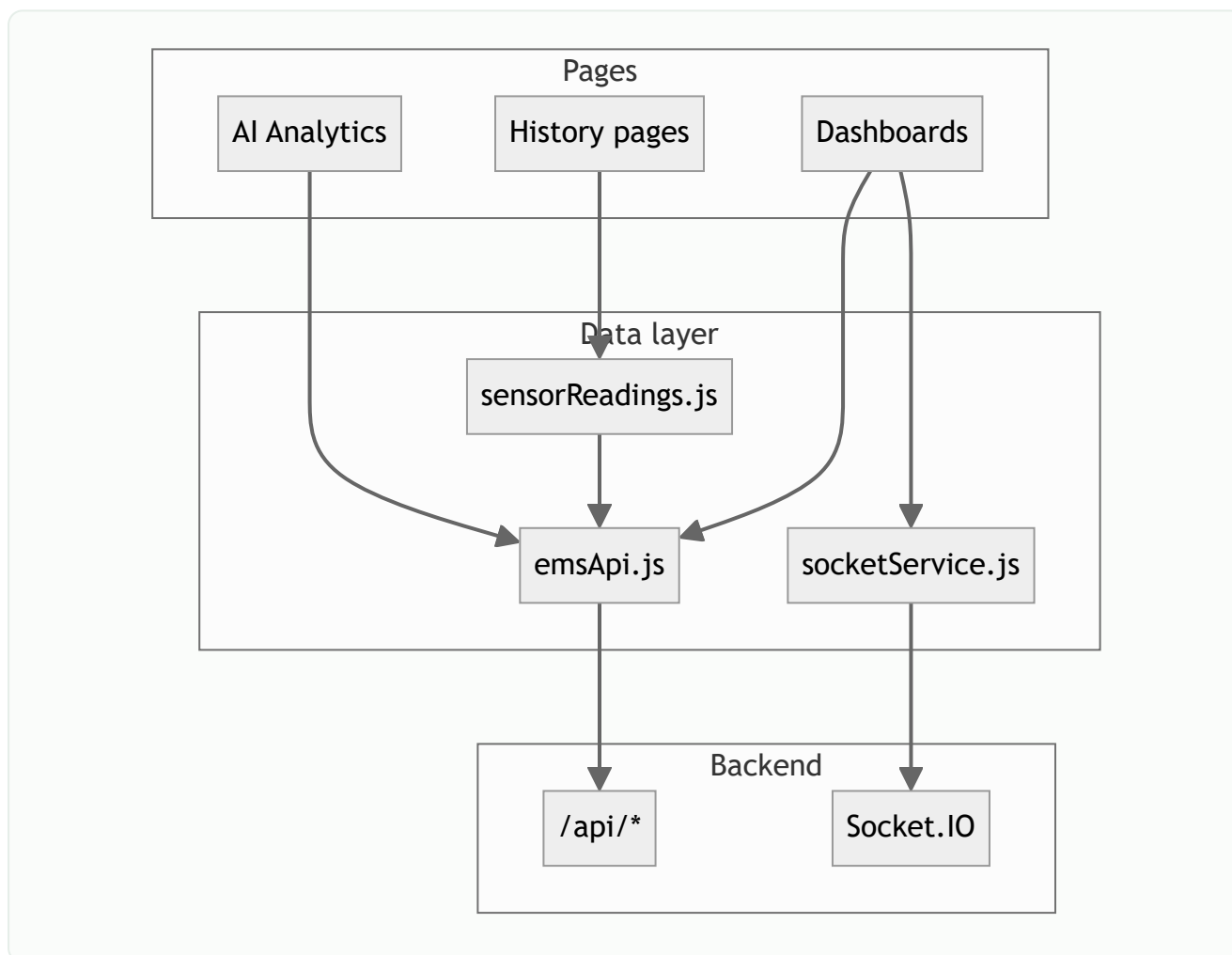
```
VITE_API_URL=https://iotbackend.yourdomain.com/api \  
VITE_SOCKET_URL=https://iotbackend.yourdomain.com \  
npm run build
```

Output: `dist/` → copied into nginx Docker image.

Docker (`web_frontend/Dockerfile`)

1. Node stage: `npm ci` + `npm run build` with build args
2. nginx stage: serve `dist/` on port 80
3. `nginx.conf` : SPA fallback (`try_files`), gzip, cache static assets

Data flow summary



Role capability matrix (web)

Feature area	Admin	Org	User
Organizations	✓	—	—
All users	✓	org only	self
Gateways / devices CRUD	✓	✓	view assigned
Device templates	✓	✓	—
Icons / products / themes	✓	—	products view
Data center / linkage	✓	—	—
Sensor & historical data	✓	✓	✓
Alarms (full config)	✓	✓	limited
Schedule tasks	✓	✓	view
AI analytics	—	—	✓
Billing (slabs / intervals)	—	—	✓
Notifications	—	—	✓

Related documents

- [Application functionality](#)
- [Backend API](#)
- [Deployment](#)
- [Architecture](#)